

## CHAPTER 06

# 06. 날퀴리(raw SQL) — 게시판을 진짜로 저장한다

---

Node.js · Express · NestJS 강의 · 2026

04장과 05장의 게시판은 서버를 끄는 순간 사라졌다. 글을 담아 둔 게 자바스크립트 배열 하나였기 때문이다. 배열은 메모리에 있고, 메모리는 프로세스가 죽으면 함께 비워진다. "게시판을 만들었다"고 부르기엔 빠진 게 하나 있었다 — **저장**이다.

이 장에서 그 빈자리를 채운다. 글을 **데이터베이스**에 넣는다. 한 번 넣은 글은 서버를 켜도, 컴퓨터를 재부팅해도 그대로 남는다. 도구는 **SQLite**, 방식은 **날퀴리(raw SQL)** — `SELECT`, `INSERT` 같은 SQL 문을 우리 손으로 직접 쓴다. 다음 장에서 배울 ORM이 이 SQL을 가리기 전에, 데이터베이스와 코드 사이에 실제로 오가는 게 무엇인지 한 번은 맨눈으로 봐 봐야 한다. 그래야 ORM이 무슨 일을 대신하는지 알 수 있다.

그리고 날퀴리를 직접 쓰면 피할 수 없이 마주치는 질문이 하나 있다. **사용자가 보낸 값을 SQL에 어떻게 끼워 넣는가**. 여기서 한 글자만 잘못 짜면 데이터베이스 전체가 털리는 **SQL 인젝션**이 열린다. 이 장의 마지막은 그 함정과, 그것을 막는 단 하나의 습관에 통째로 할애한다.

이 자료는 **개념 예습용**이다. 실제 수업에서 쓰는 파일 이름과 폴더 구성, 다루는 범위는 강사에 따라 다를 수 있다. 그래서 여기서는 파일이 아니라 **무엇을 배우는가**에 초점을 둔다.

## 이 장에서 배우는 것

- `better-sqlite3` 로 데이터베이스를 열고 SQL을 직접 실행하는 법
- `db.exec` / `db.prepare`, 그리고 실행 메서드 `.run()` · `.get()` · `.all()` 의 구분
- `CREATE TABLE`, `INSERT`, `SELECT`, `UPDATE`, `DELETE` — SQL로 하는 CRUD
- 메모리 DB( `:memory:` )와 파일 DB( `board.db` )의 차이 — "저장된다"는 것의 실체
- `CREATE TABLE IF NOT EXISTS` 와 시드(seed) 자동 삽입으로 서버를 안전하게 재실행하는 법
- `created_at DEFAULT CURRENT_TIMESTAMP` — 작성 시각을 DB가 채우게 두는 법
- **파라미터 바인딩( ? )**과 **SQL 인젝션** — 왜 값을 절대 문자열로 잊지 않는가

내용은 세 주제로 나뉘고, 04장처럼 뒤로 갈수록 개념이 쌓인다.

| 주제         | 핵심                            |
|------------|-------------------------------|
| 날퀴리 기초     | Express 없이 SQL만 — CRUD 다섯 동작  |
| 게시판 API    | 같은 SQL을 게시판 REST API로 (파일 DB) |
| SQL 인젝션 방지 | 문자열 조립의 위험 vs ? 바인딩           |

## 핵심 개념 — 데이터베이스는 "꺾다 켜도 남는 저장소"다

배열과 데이터베이스의 차이는 한 군데로 좁혀진다. **어디에 사느냐**. 배열은 프로그램의 메모리(RAM)에 살고, 데이터베이스는 디스크의 파일에 산다. 메모리는 프로세스가 끝나면 회수되지만, 파일은 그 자리에 남는다. 그래서 데이터베이스에 한 번 넣은 데이터는 서버를 꺾다 켜도, 코드를 고쳐 다시 배포해도 그대로 있다. 게시판이 게시판이라면 이 성질이 반드시 필요하다.

데이터베이스와 대화하는 언어가 **SQL**(Structured Query Language)이다. 대화는 늘 네 가지 동작 — 넣고( `INSERT` ), 읽고( `SELECT` ), 고치고( `UPDATE` ), 지운다( `DELETE` ). 04장에서 본 CRUD와 정확히 같은 네 동작이고, 이름만 SQL 문법으로 바뀐다.

| CRUD   | Express(04장)                  | SQL(06장)                      |
|--------|-------------------------------|-------------------------------|
| Create | <code>posts.push(. )</code>   | <code>INSERT INTO..</code>    |
| Read   | <code>posts.find(. )</code>   | <code>SELECT.. WHERE..</code> |
| Update | <code>post.title =..</code>   | <code>UPDATE.. SET..</code>   |
| Delete | <code>posts.splice(. )</code> | <code>DELETE FROM..</code>    |

**왜 SQLite인가** — 데이터베이스라고 하면 보통 MySQL이나 PostgreSQL처럼 따로 띄워야 하는 서버를 떠올린다. SQLite는 다르다. **파일 하나가 곧 데이터베이스**다. 설치할 서버도, 띄울 프로세스도, 연결할 주소(포트·비밀번호)도 없다. `board.db` 라는 파일이 생기고, 그 안에 테이블과 데이터가 다 들어간다. 배우기에 이만한 게 없고, 작은 서비스라면 실제 운영에도 쓰인다.

**왜 better-sqlite3 인가** — Node에서 SQLite를 다루는 드라이버다. 이름의 'better'는 동기식이라는 뜻이다. 보통 Node의 DB 호출은 콜백이나 `await` 를 거쳐 결과가 나중에 오지만, better-sqlite3 는 `.all()` 을 부르면 그 자리에서 결과 배열이 바로 돌아온다. `async / await` 없이 SQL 자체에 집중할 수 있어, 날퀴리를 배우기에 가장 군더더기가 없다.

## 준비

예제 폴더에서 `npm install` 로 의존성을 설치한다(이 장에선 `express` 와 `better-sqlite3` 두 개뿐).

**데이터베이스 파일을 미리 만들 필요는 없다**. `board.db` 도, 그 안의 테이블도, 첫 글 두 개도 전부 코드가 실행되면서 알아서 만든다. 설치 다음에 곧장 실행하면 된다. 콘솔 스크립트는 결과를 터미널에 찍고 끝나고, 게시판 API는 서버라서 그 자리에서 요청을 기다린다. 서버는 다른 터미널에서 `curl` 로 요청을 보내거나 브라우저로 접속해 확인하고, 종료는 `Ctrl+C` 다.

**네이티브 빌드 한 줄** — `better-sqlite3` 는 C/C++로 짜인 네이티브 모듈이라, 설치할 때 자기 환경에 맞춰 컴파일하는 과정이 따라온다. 대부분은 미리 만들어 둔 바이너리를 받아 와 조용히 끝나지만, 만약 설치가 컴파일에서 막히면 macOS는 Xcode Command Line Tools( `xcode-select--install` ), Windows는 빌드 도구가 깔려 있어야 한다.

## 날퀴리 기초 — Express 없이 SQL부터

서버를 붙이기 전에, SQL만 먼저 손에 익힌다. 여기서는 웹 서버를 띄우지 않는다. 그냥 실행하면 다섯 가지 SQL 동작을 차례로 수행하고 결과를 콘솔에 찍은 뒤 끝나는 콘솔 스크립트다.

```
const Database = require("better-sqlite3");

// ":memory:" = 메모리에만 존재하는 임시 DB. 프로그램이 끝나면 사라진다(연습용).
const db = new Database(":memory:");
```

`better-sqlite3` 를 불러오면 `Database` 라는 생성자가 나온다. 여기에 **파일 경로를 주면 파일 DB**가 열리고, 경로 대신 **`:memory:`** 라는 특수 문자열을 주면 **메모리 DB**가 열린다. 메모리 DB는 디스크에 아무 파일도 남기지 않고 RAM에만 존재한다. 프로그램이 끝나면 통째로 사라지므로, 매번 깨끗한 상태에서 시작하는 **연습·실험용**으로 딱 맞다. 그래서 SQL을 익히는 이 단계에선 메모리 DB를 쓴다. (반대로 게시판 API는 데이터를 남겨야 하니 파일 DB를 쓴다 — 곧 비교한다.)

## 테이블 만들기 — CREATE TABLE

```
db.exec(`
  CREATE TABLE posts (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    title TEXT NOT NULL,
    content TEXT NOT NULL,
    author TEXT,
    created_at TEXT DEFAULT CURRENT_TIMESTAMP -- 작성 시각을 DB가 자동으로 채운다
  )
`);
```

데이터를 넣기 전에, 데이터가 들어갈 **표의 모양**부터 정해야 한다. 그게 `CREATE TABLE` 이다. 자바스크립트 객체는 아무 키나 자유롭게 넣었지만, SQL 테이블은 어떤 **칸(컬럼)**이 있고 각 칸이 무슨 타입인지를 미리 못 박는다. `posts` 테이블의 다섯 칸을 보자.

- `id INTEGER PRIMARY KEY AUTOINCREMENT` — 각 행을 구별하는 고유 번호. `PRIMARY KEY` 는 "이 칸이 행의 식별자"라는 선언이고, `AUTOINCREMENT` 는 새 행이 들어올 때마다 **1, 2, 3...**으로 **DB가 알아서 번호를 매긴다**는 뜻이다. 04장에서 `nextId++` 로 직접 챙기던 일을 DB가 대신한다.

- `title TEXT NOT NULL` · `content TEXT NOT NULL` — 글자 타입( `TEXT` )이고, `NOT NULL` 이라 비어 있으면 안 된다. 빈 제목·내용을 넣으려 하면 DB가 거부한다.
- `author TEXT` — `NOT NULL` 이 없으니 작성자는 비워도 된다.
- `created_at TEXT DEFAULT CURRENT_TIMESTAMP` — 작성 시각. `DEFAULT CURRENT_TIMESTAMP` 가 핵심이다. INSERT할 때 이 값을 안 주면 **DB가 그 순간의 현재 시각을 자동으로 채운다**. 작성 시각을 코드에서 일일이 `new Date()` 로 만들어 넣지 않아도 된다. 데이터베이스를 쓰면서 생기는 첫 번째 이점이다.

`db.exec` 는 결과를 돌려받을 필요 없는 **SQL**을 그냥 실행할 때 쓴다. 테이블을 만들거나, 여러 문장을 한꺼번에 던질 때 다.

짚고 가기 — `--` 뒤는 SQL의 주석이다. 자바스크립트의 `//` 에 해당한다. 위 코드에서 `created_at` 옆 설명이 SQL 주석으로 들어가 있다.

## 쓰기 — prepare와 INSERT

```
// 2) 데이터 넣기 (INSERT) — 값은 ? 자리표시자로 넣는다(안전한 방법, 이유는 뒤에서 설명).
const insert = db.prepare(
  "INSERT INTO posts (title, content, author) VALUES (?, ?, ?)"
);
// .run()은 결과 정보(info)를 돌려준다 — info.lastInsertRowid에 방금 만들어진 행의 id가 들어 있다.
const info = insert.run("첫 번째 글", "안녕하세요", "김철수");
console.log("방금 넣은 글의 id:", info.lastInsertRowid);
insert.run("두 번째 글", "반갑습니다", "이영희");
```

여기서 `better-sqlite3` 의 두 단계 패턴이 처음 나온다. `db.prepare(sql)` 로 SQL을 미리 준비하고, 준비된 것을 실행한다.

`prepare` 가 돌려주는 객체에는 실행 메서드가 셋 있고, 용도에 따라 골라 쓴다.

| 메서드                 | 언제                            | 돌려주는 것                                      |
|---------------------|-------------------------------|---------------------------------------------|
| <code>.run()</code> | INSERT · UPDATE · DELETE (쓰기) | 결과 정보( <code>info</code> ) — 바뀐 행 수, 새 id 등 |
| <code>.get()</code> | 한 행 읽기                        | 행 객체 하나 (없으면 <code>undefined</code> )       |
| <code>.all()</code> | 여러 행 읽기                       | 행 객체들의 배열                                   |

INSERT 는 쓰기이므로 `.run()` 이다. 눈여겨볼 자리가 두 곳이다.

- **VALUES (?, ?, ?)** 의 물음표. SQL 문 안에 값을 직접 박지 않고, `?` 라는 자리표시자(**placeholder**) 만 둔다. 실제 값은 `.run("첫 번째 글", "안녕하세요", "김철수")` 처럼 따로 넘긴다. 그러면 드라이버가 첫 `?` 에 첫 인자, 둘째 `?` 에 둘째 인자... 순서대로 끼워 넣는다. 이것 **파라미터 바인딩**이라 한다. 지금은 그냥 "값은 `?` 로 넘긴다"고만 알아 두자 — 왜 반드시 이래야 하는지는 이 장 마지막의 SQL 인젝션에서 통째로 설명한다.

- `info.lastInsertRowid` . `.run()` 이 돌려주는 `info` 에는 방금 들어간 행의 `id`가 들어 있다. `AUTOINCREMENT` 로 DB가 매긴 번호를 코드가 다시 돌려받는 통로다. 뒤에서 게시판 API가 "방금 만든 글을 응답으로 돌려줄 때" 이 값을 요긴하게 쓴다.

`insert` 를 한 번 `prepare` 해 두고 두 번 `run` 한 것도 봐 두자. 같은 모양의 SQL을 여러 번 실행할 때는 준비는 한 번, 실행은 여러 번이 효율적이다.

## 읽기 — SELECT, .all()과 .get()

```
// 3) 여러 행 조회 (SELECT.. .all())
const all = db.prepare("SELECT * FROM posts").all();
console.log("전체 글:", all);

// 4) 한 행 조회 (SELECT.. .get())
const one = db.prepare("SELECT * FROM posts WHERE id = ?").get(1);
console.log("1번 글:", one);
```

읽기는 `SELECT` 다. `SELECT * FROM posts` 는 "posts 테이블에서 모든 칸( `*` )을 가져와라"는 뜻이다.

- 목록은 `.all()` — 조건 없이 전부 가져오니 행이 여러 개다. 행 객체들의 배열이 돌아온다.
- 단건은 `.get()` — `WHERE id = ?` 로 한 행만 집어내니, 행 객체 하나가 돌아온다. 같은 `SELECT` 라도 "여러 개냐 하나냐"에 따라 `.all()` 과 `.get()` 을 가른다.

`.get(1)` 의 `1` 이 `WHERE id = ?` 의 `?` 로 들어간다. 읽기에서도 값은 `?` 로 넘기는 게 똑같다.

## 수정과 삭제 — UPDATE · DELETE

```
// 5) 수정 (UPDATE)
db.prepare("UPDATE posts SET title = ? WHERE id = ?").run("제목 수정됨", 1);
console.log("수정 후 1번 글:", db.prepare("SELECT * FROM posts WHERE id = ?").get(1));

// 6) 삭제 (DELETE)
db.prepare("DELETE FROM posts WHERE id = ?").run(2);
console.log("삭제 후 전체 글:", db.prepare("SELECT * FROM posts").all());
```

- `UPDATE posts SET title = ? WHERE id = ?` — "posts 에서 `id` 가 1인 행의 `title` 을 바꿔라." `SET` 은 바꿀 값, `WHERE` 는 바꿀 대상이다. `WHERE` 를 빠뜨리면 모든 행의 제목이 한꺼번에 바뀐다 — 무서운 실수이니 늘 짚으로 본다.
- `DELETE FROM posts WHERE id = ?` — `id` 가 2인 행을 지운다. 여기서도 `WHERE` 가 생명이다. `WHERE` 없는 `DELETE FROM posts` 는 테이블을 통째로 비운다.

둘 다 쓰기이므로 `.run()` 이다. 정리하면, 읽기( `SELECT` )는 `.get()` / `.all()` , 쓰기( `INSERT` / `UPDATE` / `DELETE` )는 `.run()` — 이 한 줄이 `better-sqlite3` 사용의 거의 전부다.

이 스크립트를 실행하면 콘솔에 글이 들어가고·읽히고·바뀌고·지워지는 과정이 순서대로 찍힌다. 메모리 DB라 프로그램이 끝나면 다 사라지고, 다시 실행해도 늘 같은 결과다.

## 게시판 API — REST로 SQL 감싸기

이제 앞에서 익힌 SQL을 04장의 게시판 API에 그대로 끼워 넣는다. 달라지는 건 데이터를 어디에 두느냐 하나다. 04장은 배열, 여기는 SQLite. 라우트 구조( GET / POST / PUT / DELETE , 상태코드 200·201·400·404)는 04장과 똑같으니, 바뀐 부분에 집중하자.

```
const express = require("express");
const Database = require("better-sqlite3");
const path = require("path");

const app = express();
app.use(express.json());

// 파일 기반 DB - 이 폴더의 board.db 파일에 저장된다. (서버를 다시 켜도 데이터가 남음)
const db = new Database(path.join(__dirname, "board.db"));
```

가장 중요한 한 줄이 DB를 여는 줄이다. 앞에서는 `":memory:"` 였지만, 여기서는 **파일 경로** `path.join(__dirname, "board.db")` 다.

- `__dirname` — 지금 이 파일이 있는 폴더의 절대 경로다. `path.join(__dirname, "board.db")` 는 "이 파일과 같은 폴더의 `board.db` "가 된다. 서버를 어느 위치에서 실행하든 항상 같은 파일을 가리키게 하는 안전한 방법이다.
- 이 경로의 파일이 없으면 `better-sqlite3` 가 **새로 만든다**. 그래서 사전 준비가 필요 없다. 처음 실행하면 폴더에 `board.db` 가 생기고, 그 뒤로는 그 파일을 연다.

이게 **메모리 DB**와의 결정적 차이다. 메모리 DB는 프로그램이 끝나면 사라지지만, 파일 DB는 디스크에 남는다. 서버를 `Ctrl+C` 로 끄고 다시 켜도 어제 쓴 글이 그대로 있다. 04·05장의 인메모리 배열이 끝내 하지 못한 일을 파일 DB가 해낸다.

**짚고 가기** — `board.db` 는 `.gitignore` 에 `*.db` 로 걸려 있다. 데이터베이스 파일은 **코드가 아니라 데이터**라서, 깃으로 관리하지 않는다. 코드만 받으면 실행 시점에 각자 `board.db` 가 새로 생긴다.

## 테이블 준비 — CREATE TABLE IF NOT EXISTS

```
// 테이블이 없으면 만든다.
db.exec(`
  CREATE TABLE IF NOT EXISTS posts (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    title TEXT NOT NULL,
    content TEXT NOT NULL,
    author TEXT,
    created_at TEXT DEFAULT CURRENT_TIMESTAMP -- 작성 시각을 DB가 자동으로 채운다(인메모리 배열엔 없던 값)
  )
`);
```

앞의 CREATE TABLE 과 한 군데 다르다 — **IF NOT EXISTS**. 파일 DB는 데이터가 남기 때문에, 서버를 두 번째 실행하면 posts 테이블이 이미 있다. 그냥 CREATE TABLE 이면 "이미 있는 테이블을 또 만들려 한다"며 에러가 난다. IF NOT EXISTS 는 있으면 건너뛰고, 없을 때만 만든다. 덕분에 이 코드는 첫 실행이든 백 번째 실행이든 안전하게 통과한다. 매 실행마다 테이블이 있는지를 보장한다.

## 시드 — 처음 한 번만 샘플 데이터

```
// 처음 실행 시 샘플 데이터를 한 번만 넣는다(이미 있으면 건너뛴).
const count = db.prepare("SELECT COUNT(*) AS n FROM posts").get().n;
if (count === 0) {
  const seed = db.prepare("INSERT INTO posts (title, content, author) VALUES (?, ?, ?)");
  seed.run("첫 번째 글", "SQLite에 저장된 글입니다.", "김철수");
  seed.run("두 번째 글", "서버를 꺼도 남아 있어요.", "이영희");
}
```

테이블이 막 만들어진 첫 실행에는 글이 한 줄도 없다. 빈 게시판으로 시작하면 둘러보기 불편하니, 처음 한 번만 샘플 글(시드)을 넣는다.

- SELECT COUNT(\*) AS n FROM posts — 행이 몇 개인지 센다. COUNT(\*) 는 행 수를 돌려주고, AS n 은 그 결과 칸에 n 이라는 이름을 붙인다. .get() 으로 한 행을 받아 .n 으로 그 수를 꺼낸다.
- if (count === 0) — 글이 0개일 때만 시드를 넣는다. 두 번째 실행부터는 이미 글이 있으니 count 가 0이 아니라 이 블록을 건너뛴다. 이 조건이 없으면 서버를 켤 때마다 같은 샘플이 중복으로 쌓인다.

IF NOT EXISTS (테이블)와 if (count === 0) (데이터)이 짝을 이뤄, 몇 번을 다시 켜도 처음과 같은 상태에서 출발하게 만든다. 시드를 넣은 두 글에는 created\_at 을 주지 않았는데, DEFAULT CURRENT\_TIMESTAMP 덕분에 DB가 알아서 현재 시각을 채운다.



## 목록 조회 — ORDER BY

```
app.get("/posts", (req, res) => {
  const posts = db.prepare("SELECT * FROM posts ORDER BY id DESC").all();
  res.json(posts);
});
```

04장의 `res.json(posts)` 와 모양은 같지만, `posts` 가 배열이 아니라 **SELECT** 의 결과다. 새로 붙은 건 **ORDER BY id DESC** — "id 기준 내림차순(큰 번호부터)으로 정렬하라". 최신 글이 위로 오게 하는 흔한 처리다. 배열을 직접 `sort` 하던 일을 SQL 한 조각이 대신한다.

## 단건 조회 — 문자열 id를 그대로 넘기는 이유

```
app.get("/posts/:id", (req, res) => {
  // 참고: 04에선 Number(req.params.id)로 바꿨지만, SQLite는 id가 INTEGER라
  // "3" 같은 문자열도 알아서 숫자로 맞춰 비교한다(여기선 변환 불필요).
  const post = db.prepare("SELECT * FROM posts WHERE id = ?").get(req.params.id);
  if (!post) {
    return res.status(404).json({ message: "게시물을 찾을 수 없습니다." });
  }
  res.json(post);
});
```

04장에서는 `Number(req.params.id)` 로 꼭 숫자 변환을 해야 했다. 배열의 `find` 가 `"2" === 2` 를 `false` 로 판정하기 때문이었다. 여기서는 **변환하지 않는다**. `req.params.id` 가 `"3"` 이라는 문자열이어도 `.get("3")` 으로 그냥 넘긴다.

이유는 비교의 주체가 바뀌어서다. 자바스크립트가 비교하는 게 아니라 **SQLite가 비교한다**. `id` 칸은 `INTEGER` 타입이라, SQLite는 들어온 `"3"` 을 숫자 3으로 맞춰 비교한다. 누가 비교하느냐에 따라 같은 코드가 달라지는 좋은 예다. (그렇다고 모든 변환이 필요 없다는 뜻은 아니다 — DB-타입에 따라 다르니, 헛갈리면 숫자로 바꿔 넘겨도 무방하다.)

없으면 `.get()` 이 `undefined` 를 주므로, 04장과 똑같이 404로 답한다.

## 생성 — INSERT 후 다시 SELECT

```
app.post("/posts", (req, res) => {
  const { title, content, author } = req.body;
  if (!title || !content) {
    return res.status(400).json({ message: "title과 content는 필수입니다." });
  }
  const info = db
    .prepare("INSERT INTO posts (title, content, author) VALUES (?, ?, ?)")
    .run(title, content, author || "익명");
  // 방금 만든 행(자동 부여된 id 포함)을 다시 조회해 돌려준다.
  const created = db.prepare("SELECT * FROM posts WHERE id = ?").get(info.lastInsertRowid);
  res.status(201).json(created);
});
```

입력 검증(400)과 기본값( `author || "익명"` ), 성공 상태코드(201)는 04장과 같다. 새로운 흐름은 "넣고 → 다시 읽어서 → 돌려준다" 이다.

- INSERT 는 `.run()` 이고, 그 `info.lastInsertRowid` 에 방금 만들어진 글의 id가 들어 있다(앞에서 본 그 값이다).
- 그 id로 SELECT.. WHERE id = ? 를 다시 한다. 왜 굳이 다시 읽는가? INSERT에 우리가 넘긴 값은 `title · content · author` 뿐이다. `id` (AUTOINCREMENT)와 `created_at` (DEFAULT)은 DB가 채운 값이라 우리 손에 없다. 다시 SELECT해야 그 두 칸까지 채워진 완성된 글을 얻는다. 그래야 클라이언트가 부여된 id와 작성 시각을 바로 받아 볼 수 있다.

이 "INSERT 뒤 SELECT"는 04장에는 없던 패턴이다. 04장은 우리가 객체를 통째로 만들어 push 했으니 다시 읽을 게 없었지만, 여기서는 일부 값을 DB가 만들기 때문에 한 번 더 읽는다.

## 수정 — 보낸 필드만, SQL로

```
app.put("/posts/:id", (req, res) => {
  const post = db.prepare("SELECT * FROM posts WHERE id = ?").get(req.params.id);
  if (!post) {
    return res.status(404).json({ message: "게시물을 찾을 수 없습니다." });
  }
  const title = req.body.title === undefined ? req.body.title : post.title;
  const content = req.body.content === undefined ? req.body.content : post.content;
  const author = req.body.author === undefined ? req.body.author : post.author;
  db.prepare("UPDATE posts SET title = ?, content = ?, author = ? WHERE id = ?").run(title, content, author, req.params.id);
  res.json(db.prepare("SELECT * FROM posts WHERE id = ?").get(req.params.id));
});
```

"보낸 필드만 바꾸고 안 보낸 건 그대로"라는 04장의 원칙은 똑같다. 다만 SQL의 UPDATE 는 SET 에 적은 칸을 무조건 다 덮어쓴다. 칸 하나만 바꾸는 부분 갱신이 SQL엔 자연스럽지 않다. 그래서 요령을 쓴다.

- 먼저 `SELECT` 로 기존 글 전체를 읽는다(없으면 404).
- 각 칸을 `req.body.title === undefined ? req.body.title : post.title` 로 정한다. 요청에 그 필드가 왔으면 새 값, 안 왔으면 기존 값. 04장의 `if (title === undefined) post.title = title` 을 삼항 연산자로 옮겼다. `undefined` 로 검사하는 이유도 같다 — 빈 문자열 `""` 로 제목을 지우려는 의도를 살리려고, "값을 보냈는가"를 "값이 참인가"와 구분한다.
- 그렇게 정한 세 값으로 `UPDATE` 를 한 번 실행한다. 결국 세 칸을 다 쓰되, 안 바뀔 칸엔 원래 값을 다시 넣는 셈이다.
- 마지막으로 다시 `SELECT` 해 갱신된 글을 돌려준다(생성과 같은 이유).

`WHERE id = ?` 가 빠지면 모든 글이 같은 내용으로 덮어써진다. `UPDATE` 에 `WHERE` 는 늘 붙는다.

## 삭제 — 지우기 전에 확인

```
app.delete("/posts/:id", (req, res) => {
  const post = db.prepare("SELECT * FROM posts WHERE id = ?").get(req.params.id);
  if (!post) {
    return res.status(404).json({ message: "게시물을 찾을 수 없습니다." });
  }
  db.prepare("DELETE FROM posts WHERE id = ?").run(req.params.id);
  res.json({ message: "삭제됨", post });
});
```

- 먼저 `SELECT` 로 글이 있는지 확인한다. 없으면 404. 지운 뒤에는 그 글의 내용을 알 수 없으니, 응답에 담을 `post` 를 미리 읽어 둔다.
- 그다음 `DELETE FROM posts WHERE id = ?` 로 지운다.
- 응답으로 "삭제됨"과 함께 지워진 글의 내용을 돌려준다. 04장에서 `splice` 가 돌려주던 `removed` 를, 여기서는 미리 읽어 둔 `post` 로 대신한다.

마지막은 04장과 똑같이 3000번 포트에서 서버를 띄운다.

```
app.listen(3000, () => {
  console.log("http://localhost:3000 에서 실행 중");
});
```

서버를 띄우고 `curl` 로 글을 하나 만든 뒤, 서버를 껐다가 다시 켜고 `GET /posts` 를 해 보라. 만든 글이 그대로 있다. 이 한 번의 확인이 이 장 전체의 핵심이다.

## SQL 인젝션 방지 — 왜 값을 ? 로 넣어야 하는가

앞에서 "값은 ? 로 넘긴다"고 반복했다. 여기서는 그 규칙을 어겼을 때 무슨 일이 벌어지는지를 직접 보여 준다. 보안에서 가장 유명하고 가장 흔한 사고, **SQL 인젝션**이다. 이 역시 서버가 아니라 콘솔 스크립트로 확인한다.

```
const Database = require("better-sqlite3");
const db = new Database(":memory:");

db.exec(`
  CREATE TABLE posts (id INTEGER PRIMARY KEY, title TEXT, author TEXT);
  INSERT INTO posts (title, author) VALUES
    ('공개 글', '김철수'),
    ('또 다른 글', '이영희'),
    ('비밀 글', '관리자');
`);

const input = process.argv[2] || "김철수";
console.log(`검색어: "${input}"`);
```

연습용 메모리 DB에 글 세 개를 넣는다. 그중 '비밀 글' (작성자 '관리자')은 남에게 보이면 안 되는 글이라고 하자.

`process.argv[2]` 는 명령줄에서 받은 검색어다. 실행할 때 검색어를 인자로 주면 `input` 에 그 값이 들어간다. 이 입력을 "공격자가 검색창에 친 값"으로 생각하면 된다.

## 위험한 방법 — 문자열을 이어 붙인다

```
function searchBAD(author) {
  const sql = `SELECT * FROM posts WHERE author = '${author}'`;
  console.log(" 위험한 SQL:", sql);
  return db.prepare(sql).all();
}
```

가장 직관적이고, 가장 위험한 방법이다. 검색어를 SQL 문자열에 템플릿 리터럴로 그대로 끼워 넣는다. `author` 가 "김철수" 면 만들어지는 SQL은 이렇다.

```
SELECT * FROM posts WHERE author = '김철수'
```

정상 입력에서는 멀쩡히 동작한다. 그래서 위험을 못 느끼기 쉽다. 문제는 입력이 평범한 이름이 아닐 때 드러난다. 공격자가 검색어로 ' OR '1'='1' 를 넣으면, 위 한 줄이 이렇게 조립된다.

```
SELECT * FROM posts WHERE author = '' OR '1'='1'
```

무슨 일이 벌어졌는지 천천히 보자. 입력 앞쪽의 ' 가 `author = '` 의 따옴표를 닫아 버렸다. 그 뒤의 `OR '1'='1'` 은 입력이 아니라 **SQL 문법의 일부**가 됐다. 그리고 `'1'='1'` 은 언제나 참이다. 결국 조건 전체가 "작성자가 빈 문자열이거나, 아니면 항상 참" — 즉 모든 행에 참이 된다. WHERE가 사실상 무력화되어 '비밀 글' 을 포함한 테이블의 모든 글이 다 빠져나온다. 작성자로만 거르려던 검색이, 입력 한 줄에 전체 노출 사고로 바뀌었다.

이게 SQL 인젝션이다. 사용자 입력이 데이터가 아니라 SQL 코드로 실행되는 사고. 주석에 적혀 있듯, 같은 방식이면 `;` `DROP TABLE posts;--` 같은 입력으로 테이블을 통째로 지우는 더 심한 공격도 열린다. 입력을 코드에 직접 박는 한, 막을

방법이 없다.

## 안전한 방법 — ? 로 넘긴다

```
function searchSAFE(author) {
  return db.prepare("SELECT * FROM posts WHERE author = ?").all(author);
}
```

코드는 짧지만 차이는 결정적이다. SQL에는 값 대신 ? 만 두고, 실제 값은 .all(author) 로 따로 넘긴다. 이렇게 하면 드라이버가 **SQL의 구조와 값을 완전히 분리해서** 처리한다.

핵심은 이거다 — ? 로 넘긴 값은 무슨 내용이든 통째로 하나의 '값'으로만 취급된다. 공격자가 ' OR '1'='1' 를 넣어도, 드라이버는 그것을 "작성자 이름이 ' OR '1'='1' 인 글을 찾아라"로 해석한다. 입력 속의 따옴표나 OR 는 SQL 문법이 아니라 그냥 찾을 이름의 일부 글자일 뿐이다. 그런 이름의 작성자는 없으니 결과는 **0건**. 공격이 그대로 빗나간다.

```
console.log("\n[위험] 결과:", searchBAD(input));
console.log("[안전] 결과:", searchSAFE(input));
```

두 함수를 같은 입력으로 나란히 돌려 비교한다. 검색어를 바꿔 가며 직접 실행해 차이를 눈으로 확인하자.

```
검색어 "김철수"          → 둘 다 정상: 김철수 글만 1건
검색어 "' OR '1'='1'"    → 위험: 비밀 글까지 전부 노출 / 안전: 0건
```

정상 입력일 때는 두 방법이 똑같아 보인다. 그래서 더 위험하다 — 테스트는 통과하고, 공격에만 뚫린다. ' OR '1'='1' 를 넣는 순간 갈린다. 위험한 쪽은 비밀 글까지 다 내주고, 안전한 쪽은 0건으로 막아 낸다.

**이 장의 가장 중요한 규칙** — 사용자 입력을 SQL 문자열에 직접 잊지 마라. 언제나 ? 로 넘긴다. 앞에서 모든 값을 ? 로 넘긴 게 바로 이 때문이었다. 검색어든, 글 내용이든, id든, 사용자에게서 온 값이라면 예외 없이 ? 다.

다음 장에서 배울 ORM은 이 안전 처리를 **자동으로** 한다. 우리가 ? 를 깜빡할 일조차 없게 만든다. 하지만 ORM이 가리는 그 일이 정확히 무엇인지, 가리지 않으면 무슨 일이 나는지는 여기서 한 번 봐야 한다.

## 한눈에 보기

| better-sqlite3 호출                  | SQL                                                             | 용도                         |
|------------------------------------|-----------------------------------------------------------------|----------------------------|
| <code>db.exec(sql)</code>          | <code>CREATE TABLE</code> , 여러 문장                               | 결과 안 받는 실행                 |
| <code>.prepare(sql).run(. )</code> | <code>INSERT</code> · <code>UPDATE</code> · <code>DELETE</code> | 쓰기 ( <code>info</code> 반환) |
| <code>.prepare(sql).get(. )</code> | <code>SELECT.. WHERE..</code>                                   | 한 행 읽기                     |
| <code>.prepare(sql).all(. )</code> | <code>SELECT..</code>                                           | 여러 행 읽기                    |

| DB 종류  | 여는 법                                  | 데이터     | 어디에     |
|--------|---------------------------------------|---------|---------|
| 메모리 DB | <code>new Database(":memory:")</code> | 끝나면 사라짐 | 연습·실험   |
| 파일 DB  | <code>new Database("board.db")</code> | 디스크에 남음 | 게시판 API |

## 흔한 실수

- `UPDATE` / `DELETE` 에서 `WHERE` 를 빠뜨린다. → 모든 행이 바뀌거나 지워진다. SQL 최악의 사고이니 `WHERE` 는 늘 짚으로 본다.
- 값을 문자열로 이어 붙인다. → SQL 인젝션이 열린다. 사용자 입력은 예외 없이 `?` 로 넘긴다.
- 그냥 `CREATE TABLE` 을 쓴다. → 파일 DB는 두 번째 실행에서 "테이블이 이미 있다"며 에러. `IF NOT EXISTS` 를 붙인다.
- 시드 조건을 안 건다. → 켤 때마다 같은 샘플이 중복으로 쌓인다. `if (count === 0)` 로 처음 한 번만 넣는다.
- `.get()` 과 `.all()` 을 헷갈린다. → 한 행 자리에 배열이, 목록 자리에 객체 하나가 온다. 한 행은 `.get()` , 여러 행은 `.all()` .
- `db` 파일을 깃에 커밋한다. → 데이터는 코드가 아니다. `*.db` 는 `.gitignore` 에 둔다.

## 직접 해보기

1. 게시판 API에 작성자 검색을 더해 보자. `GET /posts?author=김철수` 로 그 작성자 글만 나오게. 힌트:  
`req.query.author` 가 있으면 `SELECT * FROM posts WHERE author = ?` 로, 없으면 전체. (값은 반드시 `?` 로!)
2. 글을 하나 `POST` 로 만들고, 서버를 꺾다 다시 켜 뒤 `GET /posts` 로 그 글이 남아 있는지 확인하자. 인메모리(O4장)와의 차이를 눈으로 본다.
3. SQL 인젝션 예제의 `searchBAD` 에 `'; DROP TABLE posts;--` 같은 입력을 넣으면 무슨 SQL이 만들어지는지 출력(위험한 SQL: )으로 확인해 보자. 안전한 쪽은 왜 아무 일도 안 일어나는지 설명해 보자.

## 정리

---

- 데이터베이스는 **꺾다 켜도 남는 저장소**다. SQLite는 파일 하나가 곧 DB이고, `better-sqlite3` 로 SQL을 직접 실행한다.
- 읽기( `SELECT` )는 `.get()` / `.all()` , 쓰기( `INSERT` · `UPDATE` · `DELETE` )는 `.run()` . `db.exec` 는 결과 안 받는 실행에 쓴다.
- 메모리 DB( `:memory:` )는 연습용, 파일 DB( `board.db` )는 데이터를 남긴다. `IF NOT EXISTS` 와 `if (count == 0)` 로 몇 번을 다시 켜도 같은 상태에서 출발한다.
- `id` (AUTOINCREMENT)와 `created_at` (DEFAULT CURRENT\_TIMESTAMP)처럼 **DB가 채우는 값**이 있어, INSERT 뒤 다시 SELECT해 완성된 글을 돌려준다.
- 가장 중요한 습관 하나 — **사용자 입력은 언제나 ? 로 넘긴다**. 문자열로 이르면 SQL 인젝션이 열린다.

다음 장(**07 · ORM**)에서는 같은 게시판을 **ORM**(Sequelize)으로 다시 만든다. 이 장에서 손으로 쓴 SQL과 ? 바인딩을 ORM이 어떻게 대신하는지, 그리고 무엇을 얻고 무엇을 잃는지 비교한다.